

A3-PS001-TC228

Android Ransomware-like Activity Early Warning in a Controlled Lab

Kenshin
manalmulla05@gmail.com
anaskhann2005@gmail.com

AIORI Hackathon 3
October 9, 2026

Problem Description

Ransomware is a growing **cybersecurity threat** that can make important files **unavailable** by rapidly **changing**, **renaming**, **deleting**, or **encrypting** them. In many cases, traditional security tools detect the attack only after several files have already been affected. This makes **early detection** important, especially when file activity suddenly increases within a short period.

This project proposes a **lightweight Android early-warning system** that detects **ransomware-like file activity** in a **controlled lab environment**. The system monitors a **designated test** folder on an Android emulator or rooted lab device and looks for unusual behavior, such as many files being modified quickly, repeated file renaming, extension changes, file replacement, or deletion of original files.

Instead of relying on **malware signatures**, the prototype uses **behavior-based rules** and a **sliding time window** to identify suspicious patterns. It also considers normal activities, such as editing documents, copying files, backups, or synchronization, to reduce **false alerts**. When suspicious activity crosses a defined threshold, the system immediately generates an **alert** and stops only the registered **simulated attack process**.

The project uses **synthetic files** and **harmless ransomware-like simulations** only. It does not use real ransomware, access files from other applications, or claim to provide **device-wide protection** on stock Android devices. The performance of the prototype will be measured using **detection delay, number of files affected before alert, precision, recall,** and **false-positive rate**.

The expected result is a simple, safe, and explainable prototype that demonstrates how **early behavior-based monitoring** can help identify ransomware-like activity before **extensive file damage** occurs.

Background

Ransomware is a type of malware that makes files inaccessible by encrypting or modifying them. It can cause data loss by rapidly changing, renaming, or deleting files. Detecting these activities early is important to reduce damage.

Traditional security approaches may rely on known malware signatures or detect threats after suspicious activity has already occurred. A behavior-based approach can instead monitor file operations and identify unusual patterns that may indicate ransomware.

The proposed Android Ransomware Early-Warning System uses a sliding time window and behavior-based detection rules to analyze file modifications, renames, and deletions. It aims to detect suspicious activity quickly, reduce false alarms from normal operations, and terminate the simulated attack process before too many files are affected.

The prototype will be evaluated in a controlled Android emulator using both normal workloads and simulated ransomware attacks. Performance will be measured using detection delay, precision, recall, and the number of files affected before detection.

Literature & RFC Review

Existing Work, Studies & Standards

- Reactive honeyfile monitoring (Gómez-Hernández & García-Teodoro, 2024): Uses Android FileObserver/inotify events to identify suspicious file-operation sequences such as rapid modification, deletion, and rename activity. This directly supports our designated-directory monitoring approach. pmc.ncbi.nlm.nih
- Real-time system-call behavioral detection (Chew et al., 2024): Demonstrates that ransomware-like behavior can be recognized through ordered file-operation patterns using finite-state machines,rather than relying on malware signatures.dl.acm
- MITRE ATT&CK Mobile – T1471: Data Encrypted for Impact: Defines ransomware-relevant impact behavior and recommends detecting abnormal bursts of file reads/writes, bulk rewrites, renames, replacements, and extension changes over short time windows. attack.mitre, attack.mitre
- Android Security Best Practices: Android sandboxing limits cross-app inspection on stock devices; therefore, our prototype is intentionally restricted to an explicitly accessible test directory in an emulator/rooted lab device. developer.android

Our approach: Combine directory telemetry, burst-rate thresholds, rename/rewrite sequence detection, and safe workload termination to provide an explainable early-warning system, without real malware or device-wide prevention claims.

Proposed Solution

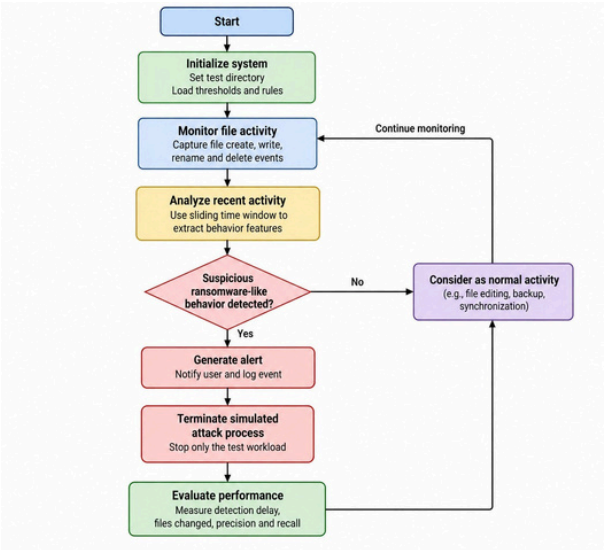
A behavior-based early-warning system for ransomware-like file activity in a controlled Android lab.

- The prototype monitors only one authorized test directory on an Android emulator or rooted lab device.
- It collects file-system events such as create, modify, write, delete, move, and rename using Android FileObserver/lab telemetry.
- A behavior engine analyzes events in a short rolling time window to identify suspicious patterns:
  - Rapid file rewrites
  - Bulk file renaming or extension changes
  - Multiple files changed within seconds
  - Rewrite → rename/delete sequences across folders
- An alert is generated only when several conditions occur together, reducing false positives from normal file editing or backup activity.
- On detection, the system logs the event details and stops only the controlled ransomware-like simulation, not other apps or the device.
- The prototype is evaluated using 2 benign workloads and 3 harmless simulations with detection delay, files changed before alert, precision, and recall as performance metrics.
- The design follows Android sandbox limitations and MITRE ATT&CK's behavior-based approach for Data Encrypted for Impact—without real malware or device-wide prevention claims.

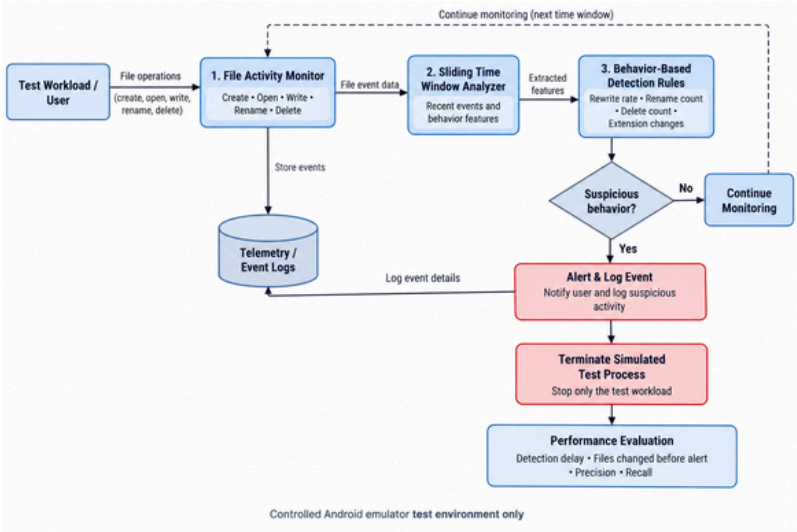
Optimization Proposed

Optimization Comparison: Android Ransomware-Like Activity Early-Warning System		
Optimization Area	**Before Optimization**	**After Optimization**
File Monitoring	Periodic basic file checks	Continuous event tracking or polling at 200 ms intervals
Detection Logic	Simple file-change detection	Sliding-window analysis of modifications, renames, and deletions
Threat Identification	Fixed threshold	Combined risk scoring based on multiple suspicious behaviors
Response Time	Alert after significant file damage	Early alert when suspicious activity crosses the detection threshold
Process Control	Manual termination	Automatic termination of the known test workload PID
False Positives	Normal file operations may trigger alerts	Benign backup and editing workloads and the detection thresholds
Performance Evaluation	Manual observation	Automated measurement of detection delay, precision, recall, and files changed before alert
Testing Reliability	Single test execution	Repeated trials across two benign workloads and three simulated attacks

Solution Architecture & Design



Data Flow Diagram



Deployment Timeline

Phase 1: Environment Setup
- Install Android Studio, configure the Android emulator, and verify ADB connectivity.
- Create the test directory and generate 100 dummy files.

Phase 2: Workload Development
- Develop two benign workloads and three simulated ransomware-like attack workloads.
- Validate file operations in the isolated test environment.

Phase 3: Detector Implementation
- Implement file monitoring and sliding-window detection.
- Configure risk thresholds, alerts, event logging, and test-process termination.

Phase 4: Integration & Testing
- Integrate the workloads with the detector.
- Test detection accuracy, false positives, response time, and process termination.

Phase 5: Evaluation & Optimization
- Run repeated experiments and calculate precision, recall, detection delay, and files changed before alert.
- Tune detection thresholds based on observed results.

Phase 6: Final Demo & Documentation
- Prepare the system demonstration, architecture diagram, README, and results presentation.
- Document limitations and future improvements.

Methodology: Iterative development → Controlled simulation → Testing → Performance evaluation → Optimization → Final demonstration.

Bibliography I

Android Developers. Android Studio and Android Emulator Documentation. https://developer.android.com/studio

Android Developers. FileObserver API Reference. https://developer.android.com/reference/android/os/FileObserver

Linux man-pages Project. inotify — Monitoring filesystem events. https://man7.org/linux/man-pages/man7/inotify.7.html

MITRE ATT&CK. Data Encrypted for Impact (T1486). https://attack.mitre.org/techniques/T1486/

Thank You for Your Attention!